

Practical 8:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc()
    int *a = malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = i + 1;

    printf("malloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\n");

    // calloc()
    int *b = calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);
```

```
printf("calloc: ");
for (i = 0; i < 5; i++)
    printf("%d ", b[i]);
printf("\n");

// realloc()
a = realloc(a, 10 * sizeof(int));

if (a == NULL)
{
    printf("realloc failed\n");
    free(b);
    return 1;
}

for (i = 5; i < 10; i++)
    a[i] = i + 1;

printf("After realloc: ");
for (i = 0; i < 10; i++)
    printf("%d ", a[i]);
printf("\n");

// free()
free(a);
free(b);

printf("Memory released successfully.\n");

return 0;
```

```
bhanu@DESKTOP-DE32HQ8:~$ cd ~
mkdir -p memory_allocation
cd memory_allocation
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ nano allocation.c
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ gcc -Wall -Wextra -o allocation allocation.c
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ ./allocation
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
Memory released successfully.
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ valgrind --version
valgrind-3.22.0
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ valgrind --leak-check=full ./allocation
==1265== Memcheck, a memory error detector
==1265== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1265== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1265== Command: ./allocation
==1265==
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
Memory released successfully.
==1265==
==1265== HEAP SUMMARY:
==1265==     in use at exit: 0 bytes in 0 blocks
==1265==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==1265==
==1265== All heap blocks were freed -- no leaks are possible
==1265==
==1265== For lists of detected and suppressed errors, rerun with: -s
==1265== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main()
{
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < SIZE; i++)
        data[i] = 10;

    printf("Before fork()\n");
    printf("PID: %d\n", getpid());
    printf("Address of data: %p\n", (void *)data);
    printf("data[0] = %d\n", data[0]);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild process\n");
    }
}
```

```

    if (pid == 0)
    {
        printf("\nChild process\n");
        printf("Child PID: %d\n", getpid());
        printf("Address of data: %p\n", (void *)data);
        printf("Before modification: data[0] = %d\n", data[0]);

        data[0] = 999;

        printf("After modification: data[0] = %d\n", data[0]);

        sleep(5);

        free(data);
        exit(0);
    }
    else
    {
        printf("\nParent process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Address of data: %p\n", (void *)data);

        sleep(2);

        printf("Parent data[0] = %d\n", data[0]);

        wait(NULL);

        free(data);
    }

    return 0;
}

```

```

bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ nano cow.c
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ gcc -Wall -Wextra -o cow cow.c
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ ./cow
Before fork()
PID: 1272
Address of data: 0x7c58acc2f010
data[0] = 10

Parent process
Parent PID: 1272
Address of data: 0x7c58acc2f010

Child process
Child PID: 1273
Address of data: 0x7c58acc2f010
Before modification: data[0] = 10
After modification: data[0] = 999
Parent data[0] = 10
bhanu@DESKTOP-DE32HQ8:~/memory_allocation$ |

```